THOMAS MORE

Reflection

Internship at Aigency

Ondrej Dobiš
Applied Computer Science

2026

Table of contents

1. Introduction

This reflection evaluates my internship project at Aigency, where I worked on Lecture Guru. In this document I reflect on what was achieved, what value the work created for the client organization and users, what still needs attention, and what I personally learned from the process.

The main theme of the internship was building an AI-supported learning and presentation functionality while continuously adapting to changing requirements. A large part of the work focused on making Lecture Guru more capable, more reliable, and easier to use.

The reflection also covers what went right and what went wrong, especially the importance of delivering high quality code output, shipping valuable features, and applying a fail-fast rule when experimental ideas did not create enough progress or product value.

2. Substantive reflection

At the start of the internship I focused on understanding the existing system and making myself productive in the development environment. This included gaining access to the company tools and repositories, configuring the local setup, creating a local database, and testing the current functionality so I could understand the application logic and data flow. This early phase was important because the later work touched many parts of the application: frontend interfaces, backend routes, database schemas, file handling, AI prompts, exports, billing, localization, and production reliability.

One of the first substantial areas I worked on was the projects, or spaces, feature. I built the basic create, read, update, and delete functionality, connected projects to chats and uploaded sources, and added personalization settings such as custom instructions, fonts, colors, stored presentations, and reusable context. This gave users and the client organization a better way to organize learning material and preserve context across conversations. Instead of each chat being isolated, projects could collect sources, preferences, and generated outputs in one place, making the system more useful for repeated work.

The project then expanded into improving the generation flow itself. I worked on a select-choices feature where the AI can ask users for important parameters such as presentation length, topic, target audience, and explanation complexity before generating output. This made the experience less automatic in a bad way and more guided: the user can either choose from suggested options or type a different answer. I also spent time on prompt engineering so the selected choices were actually respected by the model. This created value because it reduced the chance of producing a presentation that technically works but does not match the user's real needs.

The biggest delivered area was the interactive slides system. Over many weeks, Lecture Guru moved beyond static exports like PDF, PPTX, and video into code-based interactive presentations with templates, animations, AI-generated images, voiceovers, charts, quizzes, sharing, analytics, and editor support. I helped implement template selection, presentation settings, chart slots, responsive layouts, localized voiceover, AI-generated and stock images, billing for major generation actions, email delivery, and higher slide-count limits. Later I worked on the editor so authors could adjust charts, images, text, quiz content, slide layouts, and presentation settings more directly. This gave users richer output and more control after generation, which is important because AI-generated work often needs human refinement before it is useful.

A major value for both the client and users was the amount of editor and reliability work added around the generated presentations. I worked on inline editing, image cropping and replacement, chart editing, undo and redo support, autosave behavior, responsive previews, share links, viewer progress, quiz analytics, CSV exports, PDF and video exports, and completion emails. I also contributed to bilingual support in Slovak and English by replacing hardcoded interface text with translation keys and updating user-facing emails. These improvements turned the system from a demo into a real production tool that people can use.

Quality and safety were also a constant part of the project. During my work I found, investigated, and fixed many bugs. Private Cloudflare R2 source downloads needed a secure API, preview edits were missing from exports, stale OpenAI response references caused errors, cache invalidation made share links show old content, billing tier resolution identified paid users as free, voiceover tasks could crash or hang, and some public/private routes needed stricter behavior. I wrote and extended unit tests, performed manual testing, responded to automated code review feedback, fixed build issues, added smoke validation for generated decks, and applied security patches such as authorization fixes, credit-reservation fixes, and stricter postMessage origins. This work matters because advanced AI features are only valuable if the user can trust the result and if the system behaves safely in production.

The project was not completely finished in every area, but a large amount of functionality was put into use or prepared for production. Several features were merged, deployed, monitored, and refined after feedback. The more mature areas include projects, personalization, interactive slides, the editor, quizzes, sharing and analytics, localization, billing improvements, and many export-related flows. At the same time, some newer areas still need further hardening, especially Custom AI presentations, export reliability, and future generation architecture. Custom AI became a major step forward because users can describe a bespoke presentation and let an agent build it, but this kind of feature also requires continued validation, source-safety checks, rebuild reliability, and predictable editing behavior before it can be considered fully settled.

The changes compared with the original project plan were handled as part of this same iterative way of working. Because Aigency is a start-up and Lecture Guru was still in an initial product phase, ideas naturally became more concrete while they were being implemented. I did not treat the plan as something that had to be followed mechanically. Instead, I used it as a direction and adjusted the concrete work when implementation, feedback, or product priorities showed that a different shape would create more value. This is why some planned ideas became part of broader product flows, while additional reliability, security, billing, export, and editor tasks became necessary to make the delivered features usable in practice.

The feature output during the internship was high, but as the product grows, it becomes even more important to keep shared sources of truth for templates, slide capabilities, chart limits, media rules, billing behavior, cache invalidation, and export status. I would also continue investing in manual QA, automated tests for critical editor and billing flows, security review, production monitoring, and code review feedback. For AI-heavy features specifically, I would recommend keeping the generation pipeline modular and observable, so failures can be repaired or explained instead of becoming silent broken outputs for users.

3. Personal reflection

Personally, the internship meant a lot because I was able to work on a real product with real users, real production constraints, and changing requirements. It was not only a technical exercise. I had to think about how a feature should feel to the user, how it should fail, how it should be tested, and how it should fit into the existing product. That made the work more meaningful than simply completing isolated programming tasks.

I learned a great deal across the full stack. I also learned how interconnected the existing features are. A change to a generation prompt can affect editor validation; a cache invalidation bug can make a correct save look broken to the user; a billing configuration mistake can change how paid users experience the product. This helped me grow from thinking about features as separate pieces to thinking about the system as a whole.

What went right was the amount and quality of code output. Across the internship I delivered many features and improvements, but I also spent time cleaning them up, testing them, responding to review feedback, and fixing issues after they were discovered. The weekly report shows a consistent pattern: build a feature, test it manually, add or adjust automated tests where useful, fix bugs, improve the user experience, and then keep monitoring or refining it. This rhythm helped me produce useful work without treating the first implementation as automatically finished.

Another thing that went well was adapting to changing requirements. I had to shift focus quickly while still protecting the stability of existing functionality. That was challenging, but it also taught me how important it is to build in a way that can evolve. Refactoring shared logic, centralizing capability rules, and unifying behavior across templates were all examples of trying to make future changes easier instead of just adding more code on top of the old structure.

This also changed how I think about planning. In a start-up context, a plan is important because it gives direction, but it cannot predict every useful product decision before implementation starts. I learned that flexibility only works when it is combined with focus: I had to keep the main goals in mind, but also be willing to integrate an early idea into a larger workflow or shift effort toward reliability when that created more value for the product. That balance between adapting and still taking ownership of the result was one of the most useful personal lessons of the internship.

What went wrong was not one single failure, but rather a few spikes into experimental functionality that did not create enough progress or product value to justify more time. A clear example was the attempt to add richer canvas and text-editing capabilities using chenglou's pretext, faster element movement, and a richer, PowerPoint-like editing behavior. During that work it became clear that the direction was not useful enough because the richer editor functionality would likely come from Plate.js instead. Continuing to force that approach would

have taken too much time without enough benefit. Another example was the later IR-first presentation engine spike, where the idea of generating a structured intermediate representation before rendering source code was explored as a possible future architecture. It may still be useful later, but it was not something to continue immediately.

The positive side of those moments was that the fail-fast rule worked well. If something does not work, or if it is not creating enough value quickly enough, it is better not to waste time on it. Instead, the time should be moved toward something more useful that brings value to users and the client. In practice, this meant abandoning or postponing experimental work and returning to concrete improvements such as share analytics follow-ups, editor reliability, export flows, user-facing polish, and production fixes. I see this as one of the most important personal lessons of the internship: good engineering is not only about being able to build something, but also about knowing when not to keep building it.

I also faced many technical problems that required patience and structured debugging. Stale OpenAI item references, autosave validation problems, hidden cache behavior, export rendering issues, billing configuration mistakes, localization bugs, and public/private route behavior all required careful investigation. I addressed these problems by tracing root causes, testing edge cases, reading review feedback, improving validation, and making the system more explicit. These experiences improved my debugging skills and made me more comfortable working in complex codebases where the first visible symptom is rarely the real cause.

Overall, I grew most in product judgment and ownership. I became better at balancing speed with quality, building features while keeping reliability in mind, and adapting when requirements changed. I also became more aware that AI product development needs strong boundaries: prompts, tools, validation, editor rules, billing rules, and user-facing feedback all need to work together. My conclusion is that the internship was successful because it produced a lot of useful functionality for Lecture Guru, while also teaching me when to push forward, when to refactor, when to test more deeply, and when to stop an experiment so the project can keep moving in a more valuable direction. I am truly grateful for this experience.